Coding in Lyca

A practical guide to writing native programs in Lyca v0

by Viraj Shoor

Language designer and author of Lyca
Python-like syntax. LLVM-native binaries. Types you can trust.

2026 · Written by Viraj Shoor

About this book

This book teaches you how to write programs in Lyca, a small, statically typed language created by Viraj Shoor. Lyca looks like Python so you can read it at a glance: indentation, `def`, `if`, `for`, `return`. It does not behave like Python. Types are mandatory, there is no implicit coercion, and there is no garbage collector. A move-and-borrow checker keeps memory deterministic. The compiler emits LLVM IR; `clang` turns that IR into a native binary.

v0 is intentionally small. One source file, one `main`, a handful of types, structs, fixed-size arrays, and a single builtin named `print`. That is enough to write real programs, to learn the ownership rules, and to see native code come out the other end.

> — *Viraj Shoor, 2026*

Contents

1. What Lyca is

Lyca is a compiled language. You write a `.lyca` file, run the compiler, and get a native executable. There is no REPL in v0 and no virtual machine. The pipeline is lexer, parser, type checker (including ownership), LLVM IR, then `clang`.

The design goal is a language that an LLM can draft correctly on the first try because the syntax is familiar, and that a human can trust because the type system refuses to guess. If two types do not match, the program does not compile. If a value was moved, you cannot use it again. If a condition is not a `bool`, it is not a condition.

What you will need

- Node.js 18 or newer, to run the compiler
- `clang` on your `PATH` (Apple Clang or LLVM Clang)
- A text editor that inserts spaces, not tabs

`llc` is not required. Lyca writes a `.ll` file and asks `clang` to assemble and link it.

2. Install the compiler

```
git clone https://github.com/virajshoor/lyca.git
cd lyca
npm install
npm run build
npm test
```

Clone, build, and run the test suite.

The CLI is `node dist/cli/index.js`. After a successful build you can also `npm link` and run `lyca` directly. Throughout this book the long form is used so the commands work from a fresh clone.

```
node dist/cli/index.js --help
```

Expected usage is `lyca build <file.lyca> -o <output>`. That command writes LLVM IR to `output.ll` and a native binary to `output`.

3. Your first program

Create `hello.lyca`. Every Lyca program needs `def main() -> i32`. The integer you return is the process exit code.

```
def main() -> i32:
    print("Hello, World!")
    return 0
```

hello.lyca — the smallest useful Lyca program.

```
node dist/cli/index.js build hello.lyca -o hello
./hello
```

Stdout is `Hello, World!` followed by a newline. The exit code is `0`. Indentation must be spaces. Every function, including `main`, needs a return type. `print` is a builtin that takes a string (or a borrow of one) and writes it to stdout.

Fibonacci as an exit code

The bundled example `examples/fib.lyca` returns `fib(10)` from `main`. The process exit code is therefore 55 — a convenient way to check a small integer result without formatting numbers, which v0 cannot do yet.

```
def fib(n: i32) -> i32:
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

def main() -> i32:
    return fib(10)
```

```
node dist/cli/index.js build examples/fib.lyca -o /tmp/fib
/tmp/fib
echo $?
# 55
```

4. Anatomy of a Lyca file

A program is a sequence of `struct` and `def` items. There is no top-level executable code. Execution starts at `main`. Comments run from # to the end of the line. Blocks are indentation-based: a colon that introduces a block must be followed by a newline and an indented body. Lyca has no `pass`; an empty block is an error.

```
# comments look like this

struct Pair:
    a: i32
    b: i32

def sum(p: Pair) -> i32:
    return p.a + p.b

def main() -> i32:
    let p: Pair = Pair { a: 3, b: 4 }
    return sum(p)
```

`sum` takes `Pair` by value, so `p` is moved into the call and cannot be used afterward. Chapter 12 covers that rule in full.

5. Types and literals

Every binding, parameter, and return value has an explicit type. The compiler does not infer function signatures. Integer and float literals take the type of their context; there is no other inference, and no implicit conversion between named types.

```
let a: i32 = 10
let b: i64 = 10          # the literal becomes i64 from context
let x: f64 = 1.5
let y: f32 = 1.5
let ok: bool = true
let s: string = "hi\n"
let a3: [i32; 3] = [1, 2, 3]
```

Copy types can be used freely: `i32`, `i64`, `f32`, `f64`, `bool`, and `&T`. Move types cannot: `string`, arrays, and structs. `[i32; 2]` and `[i32; 3]` are different types. User structs are nominal: two structs with the same fields are still different types.

String literals use double quotes. Legal escapes are `\n`, `\t`, `\\`, and `\"`. Strings cannot span lines.

No coercion

Mixing `i32` and `i64` is always an error. Returning `true` from `main` is a type mismatch. `if 1:` is illegal because only `bool` is a condition. There are no casts in v0. Pick one type and stay on it.

6. Variables and mutability

Every `let` needs a type annotation. Bindings are immutable unless you write `let mut`. Assignment to a plain `let` is `LYC217`.

```
def main() -> i32:
    let x: i32 = 1
    let mut y: i32 = 2
    y = y + 1
    return x + y
```

There is no `+=`. Write `y = y + 1`. You also cannot mutate a value while a borrow of it is live; that is `LYC220`.

7. Functions

Parameter types and the return type are mandatory. Recursion is allowed. There are no methods, no closures, and no first-class function values in v0. Call syntax is `name(args)`.

```
def add(a: i32, b: i32) -> i32:
    return a + b

def abs(x: i32) -> i32:
    if x < 0:
        return -x
    return x
```

Every path through a function must `return` a value (`LYC208`). An `if` without `else` does not cover the rest of the function, which is why `abs` still needs a `return x` after the `if`.

main is special

`main` takes no parameters and must return `i32`. Missing `main` is `LYC214`. A `main` with parameters or a non-`i32` return is `LYC215`. On Unix the shell only reports exit codes 0 through 255, so return small integers from `main` or print a message and return 0.

8. Control flow

if / elif / else

The condition must be `bool`. There is no truthiness. Comparisons do not chain: `0 < n < 10` is a type error because `(0 < n)` is `bool`. Write `n > 0 and n < 10`.

```
if n == 0:
    return 1
elif n == 1:
    return 2
else:
    return 3
```

while

```
let mut i: i32 = 0
while i < 10:
    i = i + 1
```

for-range

`for name in start..end:` iterates `i32` values in the half-open interval `[start, end)`. The name is a mutable `i32` local for the loop body. There is no `break` or `continue` in v0.

```
let mut s: i32 = 0
for i in 0..4:
    s = s + i
# s == 6
```

9. Operators

Arithmetic requires the same numeric type on both sides: `+ - * / %`. Remainder is integer-only. Comparison: `== != < > <= >=`. Ordering is for numbers; equality also works on `bool`. Boolean operators are `and`, `or`, and `not`, with short-circuit `and` / `or`. Unary `-` negates numbers.

There is no `+` for strings and there are no bitwise operators. To borrow a local, write `&x` where `x` is a variable name.

10. Structs

Structs are records. No methods, no inheritance, no `self`. Field order in a literal does not have to match the definition; every field must appear exactly once.

```
struct Point:
    x: i32
    y: i32

def origin() -> Point:
    return Point { x: 0, y: 0 }

def main() -> i32:
    let mut p: Point = origin()
    p.x = p.x + 1
    return p.x
```

Reading a Copy field such as `p.x` does not move `p`. Reading a move field marks the whole struct as moved. You cannot write `&p.x` in v0; borrow the whole local, or copy a Copy field out.

Pass by borrow to keep the value

```
struct Pair:
    a: i32
    b: i32

def sum(p: &Pair) -> i32:
    return p.a + p.b

def main() -> i32:
    let p: Pair = Pair { a: 3, b: 4 }
    let s: i32 = sum(&p)
    return s + p.a
```

Exit code 10. p stays alive after sum(&p).

11. Arrays

Arrays have a fixed length that is part of the type. The index expression must be `i32`. There are no slices in v0.

```
def main() -> i32:
    let mut a: [i32; 4] = [1, 2, 3, 4]
    a[0] = 10
    let mut s: i32 = 0
    for i in 0..4:
        s = s + a[i]
    return s
```

Exit code 19. Indexing Copy elements does not move the array.

12. Ownership and borrowing

This is the part that is not Python. Copy types can be assigned and passed without invalidating the source. Move types are moved on use as a value.

```
let a: i32 = 1
let b: i32 = a          # a is still usable

let s: string = "hi"
let t: string = s       # s is moved; using s is LYC218
```

Borrow instead of moving. `&T` is a shared, immutable borrow. `&x` is only legal when `x` is a variable name.

```
def main() -> i32:
    let s: string = "hi"
    let r: &string = &s
    print(r)
    print(s)            # still valid
    return 0
```

The five rules

- **No use after move.** Once a move-type binding is moved, any use is LYC218.
- **No borrow after move.** `&s` after a move is LYC223.
- **No move while borrowed.** If a live `&T` points at s, moving s is LYC219.
- **No mutate while borrowed.** Assigning to s while borrowed is LYC220.
- **Immutability.** `let x` cannot be assigned; `let mut x` can, subject to the previous rule.

v0 does not do non-lexical lifetimes. A borrow created by `let r: &T = &s` lasts until `r` goes out of scope — the end of the current indented block — not until last use. Temporary borrows at a call site last for that statement only. There is no `&mut T`; mutation goes through the owned `let mut` binding when no borrows are live.

```
def main() -> i32:
    let mut s: string = "a"
    if true:
        let r: &string = &s
        print(r)
        # s = "b"       # LYC220, r is still in scope
    s = "b"             # ok, r is gone
    print(s)
    return 0
```

Do not return `&T` from user functions in v0. The only safe borrows to return would have to outlive the function; v0 has no such globals besides string literals, and the checker is not a full lifetime analysis.

print does not consume

`print` takes `&string`. If you pass an owned `string`, the compiler inserts a borrow for that statement. That is why you can print the same string twice.

```
def main() -> i32:
    let s: string = "ok"
    print(s)
    print(s)
    return 0
```

13. Printing and exit codes

The only builtin is `def print(s: &string) -> i32`. It writes `s` plus a newline via libc `puts` and returns 0. You cannot `print(n)` for an integer. There is no `str(n)` and no string concatenation.

Two patterns cover v0 I/O. For a small integer result, return it from `main` and inspect `$?`. For a human-readable result, print a literal and return 0.

14. Worked programs

Each program below is complete. Compile with `node dist/cli/index.js build file.lyca -o /tmp/out` and run `/tmp/out`.

Factorial

```
def fact(n: i32) -> i32:
    let mut acc: i32 = 1
    let mut i: i32 = 1
    while i <= n:
        acc = acc * i
        i = i + 1
    return acc

def main() -> i32:
    return fact(5)
```

Exit code 120.

Sum a range

```
def main() -> i32:
    let mut s: i32 = 0
    for i in 0..10:
        s = s + i
    return s
```

Exit code 45. The range 0..10 yields 0 through 9.

GCD

```
def gcd(a: i32, b: i32) -> i32:
    let mut x: i32 = a
    let mut y: i32 = b
    while y != 0:
        let t: i32 = y
        y = x % y
        x = t
    return x

def main() -> i32:
    return gcd(48, 18)
```

Exit code 6.

Manhattan distance

```
struct Point:
    x: i32
    y: i32

def manhattan(p: Point) -> i32:
    let mut ax: i32 = p.x
    if ax < 0:
        ax = -ax
    let mut ay: i32 = p.y
    if ay < 0:
        ay = -ay
    return ax + ay

def main() -> i32:
    let p: Point = Point { x: 3, y: -4 }
    return manhattan(p)
```

Exit code 7. manhattan takes Point by value, so p is moved.

FizzBuzz of one number

```
def fizzbuzz(n: i32) -> i32:
    if n % 15 == 0:
        print("FizzBuzz")
        return 0
    if n % 3 == 0:
        print("Fizz")
        return 0
    if n % 5 == 0:
        print("Buzz")
        return 0
    print("plain")
    return n

def main() -> i32:
    return fizzbuzz(15)
```

Prints "FizzBuzz" and exits 0.

Powers of two by recursion

```
def pow2(n: i32) -> i32:
    if n == 0:
        return 1
    return 2 * pow2(n - 1)

def main() -> i32:
    return pow2(3)
```

Exit code 8. Recursion is legal. Prefer a loop when you are accumulating; v0 has no tail-call guarantee.

15. Reading compiler errors

Every diagnostic has a code `LYCnnn`, a message, a `file:line:col` span, the source line, a caret, and sometimes a hint. There are no warnings in v0; anything the compiler prints is fatal. If type checking fails, nothing is written.

```
error[LYC207]: type mismatch: expected i32, found bool
 --> hello.lyca:2:12
  |
2 |     return true
  |            ^^^^
  |
  = hint: Lyca does not coerce types
```

Fix the source at that location and rebuild. Common codes while you learn: `LYC003` tab character, `LYC207` type mismatch, `LYC208` missing return, `LYC217` assign to immutable `let`, `LYC218` use after move, `LYC220` mutate while borrowed. The full list lives in `docs/error-reference.md`.

16. What v0 cannot do yet

If you are arriving from Python or JavaScript, these are the gaps that surprise people. They are not bugs; they are the current language.

- No printing of integers: `print` is `&string` only.
- No string concatenation and no interpolating strings.
- No chained comparisons, no truthiness, no `pass`.
- No `break` or `continue`.
- No mixing integer widths, no casts.
- No methods, classes, inheritance, or first-class functions.
- No modules, imports, or multi-file programs.
- No heap types, slices, generics, or `&mut T`.
- No REPL.

The ownership rules exist so heap types can be added later without a garbage collector. String literals live in static storage; the type system still treats `string` as move-only so the same rules will apply when heap strings exist. Structs and arrays are stack-allocated and copied or moved as whole values.

17. A one-page cheatsheet

```
# build
node dist/cli/index.js build file.lyca -o out

# skeleton
def main() -> i32:
    print("hello")
    return 0

# bindings
let x: i32 = 1
let mut y: i32 = 2
y = y + 1

# types
i32 i64 f32 f64 bool string [T; N] StructName &T

# control
if cond: ... elif cond: ... else: ...
while cond: ...
for i in start..end: ...     # half-open i32 range

# operators
+ - * / %    == != < > <= >=    and or not    -x    &x

# ownership
# Copy:  i32 i64 f32 f64 bool &T
# Move:  string, arrays, structs
print(s)                     # borrows, does not move
let t: string = s            # moves s
```

About the author

Viraj Shoor designed Lyca and wrote this book. The language, the compiler, the documentation, and the examples in this repository are his work. Lyca is released under the MIT License. Copyright (c) 2026 Viraj Shoor.

The source lives at `https://github.com/virajshoor/lyca`. Bug reports and small patches are welcome. Read the language tour and error reference before sending a change; tests live in `tests/` and should cover both a valid program and the diagnostic you care about.

Coding in Lyca

Written by Viraj Shoor

Lyca v0 · 2026